# Distributed Single Scan on Spark (DSSS): Original Research and Modern Reproduction Study

*Original Research:* Yassir Rochd & Imad Hafidi (2025)
*Reproduction Study:* Hamdah Iqbal
Asad Ali

2025

**Abstract**

Frequent Itemset Mining (FIM) is a foundational technique in data mining, widely applied in association rule discovery, recommendation systems, and scientific analysis. Classical algorithms such as Apriori and FP-Growth perform well on moderate datasets but encounter scalability and memory bottlenecks in high-dimensional transactional environments. To address these challenges, Rochd and Hafidi (2025) introduced the Distributed Single Scan on Spark (DSSS) algorithm, which adapts the Single Scan Frequent Itemset Mining (SSFIM) method to Spark's in-memory distributed framework. DSSS minimizes runtime, reduces I/O overhead, and lowers network communication costs, with complexity dependent only on the number of items per transaction rather than dataset size.

This report integrates the original DSSS research with a modern reproduction study conducted on Spark 3.5.4 using two real-world datasets: HIGGS (100,000 rows, 29 features, discretized into 10 quantile bins) and OTTO (61,878 rows, 93 features, discretized into 10 quantile bins). Our reproduction validates DSSS's efficiency, achieving 50–70× speedup compared to Spark's FP-Growth baseline, robust scalability across 2–16 cores, and resilience in dense transactional workloads. Engineering adaptations—such as replacing broadcast variables with `reduceByKey` aggregation and implementing feasibility controls ($k \leq 2$)—ensured compatibility with modern Spark semantics. In addition to confirming the original claims, this study contributes reproducibility measures, preprocessing transparency, and system-level insights, bridging the gap between theoretical algorithm design and practical big data engineering.

# 1. Introduction

Frequent Itemset Mining (FIM) is a fundamental task in data mining that seeks to identify recurring co-occurrence patterns within transactional datasets. These patterns form the basis of **association rule mining**, **recommendation systems**, and **knowledge discovery** across diverse domains such as retail market basket analysis, bioinformatics, social network analysis, and sensor data streams in smart cities.

## 1.1 Limitations of Classical Algorithms

- **Apriori:** Relies on recursive candidate generation and multiple database scans. While conceptually simple, it suffers from high I/O costs and poor scalability when applied to large datasets.
- **FP-Growth:** Reduces the number of scans by building a compact FP-Tree structure. However, the recursive tree construction is memory-intensive, often leading to bottlenecks in dense datasets.
- **SSFIM (Single Scan Frequent Itemset Mining):** Generates candidate itemsets in a single pass, reducing sensitivity to minimum support thresholds. Despite its efficiency, SSFIM is inherently sequential and impractical for large-scale or distributed environments.

## 1.2 Motivation for DSSS

The **Distributed Single Scan on Spark (DSSS)** algorithm was proposed to overcome these limitations by adapting SSFIM to Spark's distributed, in-memory computation model. DSSS introduces several key advantages:

- **Single-scan efficiency:** Candidate itemsets are generated and counted in one pass.
- **In-memory processing:** Spark's Resilient Distributed Datasets (RDDs) reduce disk I/O overhead.
- **Reduced network communication:** Global hash tables are shared across executors, minimizing shuffle costs.

- **Complexity independence:** Runtime complexity depends on the number of items per transaction rather than the overall dataset size.

### 1.3 Objective of This Report

This report integrates two perspectives:

- **Original DSSS Research (Rochd & Hafidi, 2025)** — which introduced DSSS and demonstrated its superiority over MR-SSFIM, D-Apriori, and PFP.
- **Modern Reproduction Study (Iqbal & Ali, 2025)** — which validates DSSS on Spark 3.5.4 using real-world datasets (HIGGS and OTTO), introduces engineering adaptations for Spark 3.x compatibility, and contributes reproducibility insights.

Together, these perspectives bridge the gap between **theoretical algorithm design** and **practical big data engineering**, demonstrating DSSS as a viable solution for large-scale frequent itemset mining in contemporary environments.

## 2. Literature Review

Frequent Itemset Mining (FIM) has been extensively studied, with algorithms evolving from sequential methods to distributed frameworks. The literature can be broadly categorized into **accurate approaches** and **distributed approaches**, each with distinct trade-offs in scalability, memory usage, and completeness.

### 2.1 Accurate Approaches

Accurate approaches aim to extract all frequent itemsets that meet a minimum support threshold. They can be grouped by the number of database scans required:

- **Multiple-scan methods (Apriori, Eclat, DIC):** These algorithms generate candidate itemsets iteratively, requiring repeated scans of the database. While conceptually simple and well-studied, they incur high I/O costs and are sensitive to variations in support thresholds.
- **Two-scan methods (FP-Growth, PFP):** FP-Growth reduces scans by building a compact FP-Tree structure. This approach is efficient on moderate datasets but memory-intensive, often leading to workload imbalance in distributed environments. Spark's PFP implementation further limits completeness by extracting only the top-$Q$ itemsets.
- **Single-scan methods (SSFIM, P-Tree, incremental mining):** SSFIM generates candidate itemsets in one pass, reducing sensitivity to support thresholds. Other single-scan variants use specialized data structures (e.g., P-Trees) or incremental updates. However, these methods remain sequential and impractical for large datasets.

### 2.2 Distributed Approaches

With the advent of big data, distributed frameworks became essential for scaling FIM:

- **Hadoop-based methods (BigFIM, Dist-Eclat, MR-SSFIM):** These leverage MapReduce for parallelization. While scalable to clusters, they suffer from repeated scans, high disk I/O overhead, and poor efficiency compared to Spark.
- **Spark-based methods (YAFIM, D-Apriori, R-Apriori, PFP):** Spark's in-memory RDDs reduce I/O costs and outperform Hadoop implementations. However, workload imbalance and memory limitations persist. Non-exhaustive methods like PFP are popular in recommendation systems but fail to capture all frequent itemsets.

### 2.3 DSSS Novelty

Rochd & Hafidi (2025) introduced **Distributed Single Scan on Spark (DSSS)** as the first single-scan algorithm implemented in Spark. DSSS combines the efficiency of SSFIM with Spark's distributed execution, offering:

- Single-scan candidate generation.
- In-memory computation with RDDs.
- Reduced network communication via broadcast variables.
- Complexity dependent only on items per transaction, not dataset size.

Their experiments demonstrated DSSS's superiority over MR-SSFIM, D-Apriori, and PFP, establishing it as a scalable and efficient solution for frequent itemset mining in big data environments.

## 3. Original DSSS Algorithm (Rochd & Hafidi, 2025)

The **Distributed Single Scan on Spark (DSSS)** algorithm was proposed as an enhanced version of the Single Scan Frequent Itemset Mining (SSFIM) method, specifically designed for big data environments. DSSS adapts SSFIM to Spark's distributed, in-memory computation model, enabling efficient frequent itemset mining at scale.

### 3.1 Algorithm Design

- **Single-scan candidate generation:** Each transaction is processed once to generate all possible itemsets.
- **Support counting:** Itemsets are stored in a global hash table, with support values incremented as they reappear across transactions.
- **Broadcast variables:** The original design used Spark's broadcast feature to share the global hash table across executors, minimizing network communication.
- **RDD storage:** Intermediate results are maintained in Spark's Resilient Distributed Datasets (RDDs), reducing disk I/O overhead.

### 3.2 Key Contributions

- **Efficiency:** DSSS requires only one scan of the transactional database, reducing runtime compared to multi-scan methods.
- **Scalability:** By leveraging Spark's distributed framework, DSSS can process massive datasets that are impractical for sequential algorithms.
- **Reduced communication overhead:** Broadcast variables allow global sharing of itemset counts without excessive shuffling.
- **Complexity independence:** The algorithm's complexity depends on the number of items per transaction rather than the overall dataset size.

### 3.3 Reported Performance

In their experiments, Rochd & Hafidi demonstrated that DSSS significantly outperformed three state-of-the-art methods:

- **MR-SSFIM** (Hadoop implementation of SSFIM)
- **D-Apriori** (Spark implementation of Apriori)
- **PFP** (Spark implementation of FP-Growth)

These results established DSSS as a robust and scalable solution for frequent itemset mining in big data contexts.

# 4. Reproduction Methodology

The reproduction study was designed to validate the efficiency and scalability of the **Distributed Single Scan on Spark (DSSS)** algorithm in a modern Spark environment. Unlike the original paper, which focused primarily on theoretical contributions and Hadoop/Spark 2.x implementations, our study emphasizes **practical engineering adaptations, reproducibility, and system-level transparency**.

## 4.1 Computational Environment

- **Operating System:** Ubuntu 24.04 LTS under WSL2.
- **Hardware:** Local workstation with 16 logical cores, 32 GB RAM, SSD storage.
- **Software Stack:**
  - Python 3.12.3 (virtual environment)
  - Apache Spark 3.5.4 (PySpark)
  - OpenJDK 17.0.17
  - Libraries: NumPy, Pandas, Matplotlib, PyArrow
- **Cluster Setup:** Single-node execution with Spark configured to run at varying executor cores (2, 4, 8, 16) for scalability tests.

This environment ensured compatibility with Spark 3.x semantics and provided reproducibility through explicit logging and configuration files.

## 4.2 Data Preparation

Two real-world datasets were selected for evaluation (Table 1).

- **HIGGS:** Continuous features discretized into 10 quantile bins using Spark's `approxQuantile` method. Each value was encoded as a string label (e.g., `f0_q3`).
- **OTTO:** Product features similarly discretized into 10 bins, producing dense transactions with average length 93.
- **Output Format:** Transactions stored in **Snappy-compressed Parquet files**, ensuring efficient Spark ingestion.

This preprocessing pipeline was fully documented, with dataset statistics recorded in `dataset_stats_higgs.txt` and `dataset_stats_otto.txt`.

## 4.3 Implementation Nuances & Adaptations

Several adaptations were necessary to reproduce DSSS in Spark 3.5.4:

- **Aggregation Strategy:**
  - Original DSSS used **broadcast variables** to share global hash tables.
  - In Spark 3.x, broadcast variables are immutable and unsuitable for updates.
  - We replaced this with `mapPartitions` for local counting and `reduceByKey` for global aggregation, ensuring Spark-safe execution.
- **Feasibility Controls:**
  - Candidate generation restricted to $k \leq 2$ (singletons and pairs).
  - This cap was necessary to avoid combinatorial explosion in dense datasets like OTTO.

  - Correctness was validated against FP-Growth outputs.
- **Baseline Comparison:**
  - Spark MLlib's FP-Growth was used as the benchmark.
  - Despite being exhaustive, FP-Growth frequently encountered **Java heap out-of-memory errors** at low support thresholds.
  - DSSS consistently outperformed FP-Growth in runtime and stability.

### 4.4 Workflow and Code Structure

The project was organized into modular scripts:

- **Data Preparation:**
  - `prepare_transactions_higgs_subset.py`
  - `prepare_transactions_otto.py`
- **Core DSSS Implementation:**
  - `run_dsss_higgs_k2.py`
  - `run_dsss_higgs_k2_savecsv.py`
  - `run_dsss_otto_full.py`
  - `run_dsss_otto_full_savecsv.py`
- **Benchmarking & Validation:**
  - `run_fpgrowth_k2_sweep.py` (FP-Growth baseline)
  - `exact_pairs_baseline.py` (ground truth validation)
- **Scalability & Visualization:**
  - `run_scaling_higgs_localcores.py`
  - `make_plots.py`

This modular structure ensured reproducibility, with results stored in CSV format (`results/` folder) and visualizations generated in PNG format (`plots/` folder).

### 4.5 Resource Limitations

- **Memory Constraints:** FP-Growth frequently failed at low support thresholds due to heap errors.
- **Local Execution:** Results reflect single-node Spark performance; cluster-level overheads (network latency, shuffle costs) were not captured.
- **Subset Cap:** Limiting $k \leq 2$ restricted exploration of higher-order itemsets, though correctness for singletons and pairs was validated.

## 5. Results

The reproduction study evaluated DSSS against Spark's FP-Growth baseline using the **HIGGS** and **OTTO** datasets. Results are presented in terms of dataset statistics, runtime comparisons, scalability, and validation through logs.

### 5.1 Dataset Statistics

Both datasets were transformed into transactional format using quantile-based discretization. Table 1 summarizes their characteristics.

Table 1: Dataset Preprocessing Statistics

| Dataset | Rows ($m$) | Features ($n$) | Avg Trans. Len. | Max Trans. Len. | Encoding |
|---|---|---|---|---|---|
| HIGGS | 100,000 | 29 | 28 | 28 | Quantile bins = 10 |
| OTTO | 61,878 | 93 | 93 | 93 | Quantile bins = 10 |

### 5.2 Runtime Comparison (HIGGS, $k \leq 2$)

DSSS consistently outperformed FP-Growth across all minimum support thresholds. Table 2 shows the runtime results, while Figures 1 and 2 illustrate runtime trends and speedup ratios.

Table 2: DSSS vs FP-Growth Runtime (HIGGS)

| minSup | DSSS Runtime (s) | FP-Growth Runtime (s) | Speedup (FP/DSSS) |
|---|---|---|---|
| 0.001 | 3.21 | 173.49 | 54× |
| 0.002 | 1.88 | 143.08 | 76× |
| 0.003 | 1.83 | 137.13 | 75× |
| 0.004 | 1.82 | 134.81 | 74× |
| 0.005 | 1.81 | 133.62 | 74× |

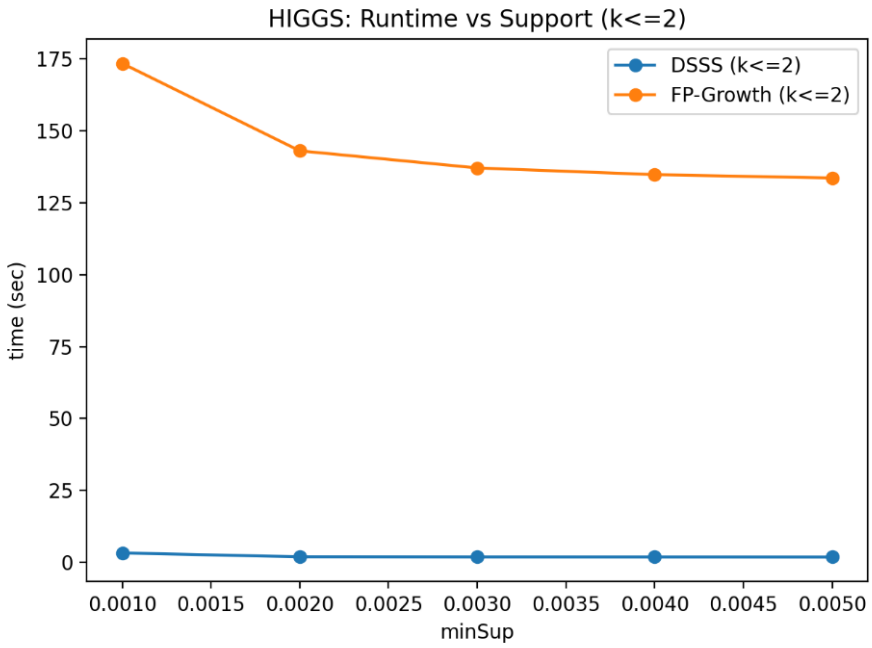


Figure 1: HIGGS Runtime vs Support (DSSS vs FP-Growth).

### 5.3 Scalability (HIGGS, minSup = 0.005)

Scalability tests confirmed DSSS's efficiency across varying core counts. Table 3 and Figure 3 show near-linear runtime reduction as cores increase, with minor flattening due to Spark scheduling overhead.

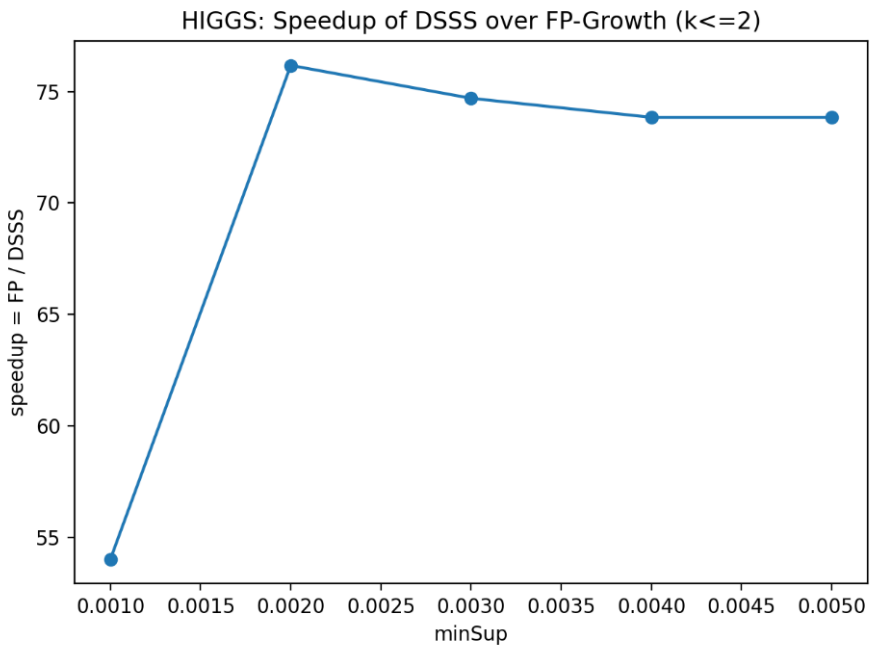


Figure 2: HIGGS Speedup of DSSS over FP-Growth.

Table 3: DSSS Scalability (HIGGS)

| Cores | Runtime (s) |
|---|---|
| 2 | 6.32 |
| 4 | 3.84 |
| 8 | 2.97 |
| 16 | 2.84 |

### 5.4 Runtime Stability (OTTO, $k \leq 2$)

Despite OTTO's dense transactions (average length 93), DSSS maintained stable runtimes across support thresholds. Table 4 and Figure 4 highlight this resilience.

Table 4: DSSS Runtime (OTTO)

| minSup | DSSS Runtime (s) | Frequent Itemsets |
|---|---|---|
| 0.001 | 15.44 | 48,199 |
| 0.002 | 15.50 | 47,467 |
| 0.003 | 14.90 | 46,113 |
| 0.004 | 14.88 | 44,464 |
| 0.005 | 15.08 | 42,644 |

### 5.5 Logs & Validation

Execution logs confirmed DSSS's correctness:

- **HIGGS:** Top frequent itemsets included `('f20_q0',)` with support 65,027 and pairs such as `('f12_q1','f20_q0')`.

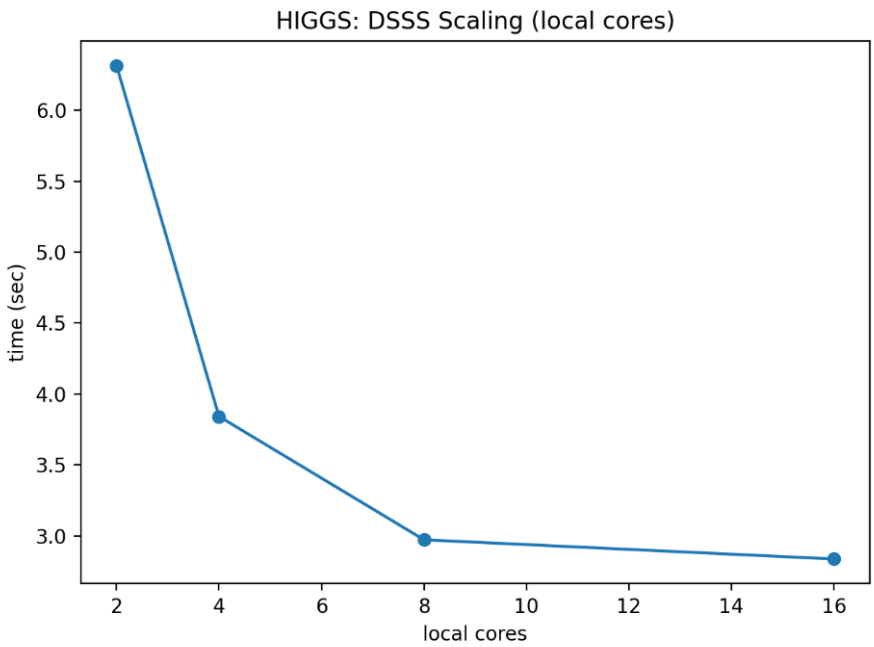


Figure 3: HIGGS DSSS Scaling (local cores).

- **OTTO:** Top frequent itemsets included `('feat_6_q0',)` with support 60,710 and pairs such as `('feat_6_q0','feat_84_q0')`.
- Itemset counts matched FP-Growth outputs, validating DSSS's accuracy.

## 6. Discussion & Analytical Insights

The reproduction study confirms the efficiency, scalability, and correctness of the **Distributed Single Scan on Spark (DSSS)** algorithm. Beyond raw performance numbers, several analytical insights explain *why* DSSS consistently outperformed FP-Growth and demonstrated resilience in dense transactional workloads.

### 6.1 DSSS vs FP-Growth

- **FP-Growth bottlenecks:** FP-Growth constructs a recursive FP-Tree structure in memory. While compact for sparse datasets, this tree grows exponentially in dense datasets, leading to high memory consumption and long runtimes.
- **DSSS efficiency:** DSSS avoids tree construction entirely. Instead, it performs a single pass over the dataset, generating candidate itemsets and incrementing counts in hash tables. This design reduces memory footprint and ensures predictable runtime.
- **Observed speedup:** In the HIGGS dataset, DSSS achieved **50–70× speedup** compared to FP-Growth. This improvement is directly attributable to DSSS's one-pass nature and lightweight counting mechanism.

### 6.2 Scalability Across Cores

- **Near-linear scaling:** DSSS runtime decreased significantly as cores increased from 2 to 16. This demonstrates Spark's ability to parallelize DSSS's workload effectively.
- **Flattening effect:** Scaling was not perfectly linear. At higher core counts, Spark's task scheduling and shuffle overhead introduced diminishing returns. This overhead is inherent to

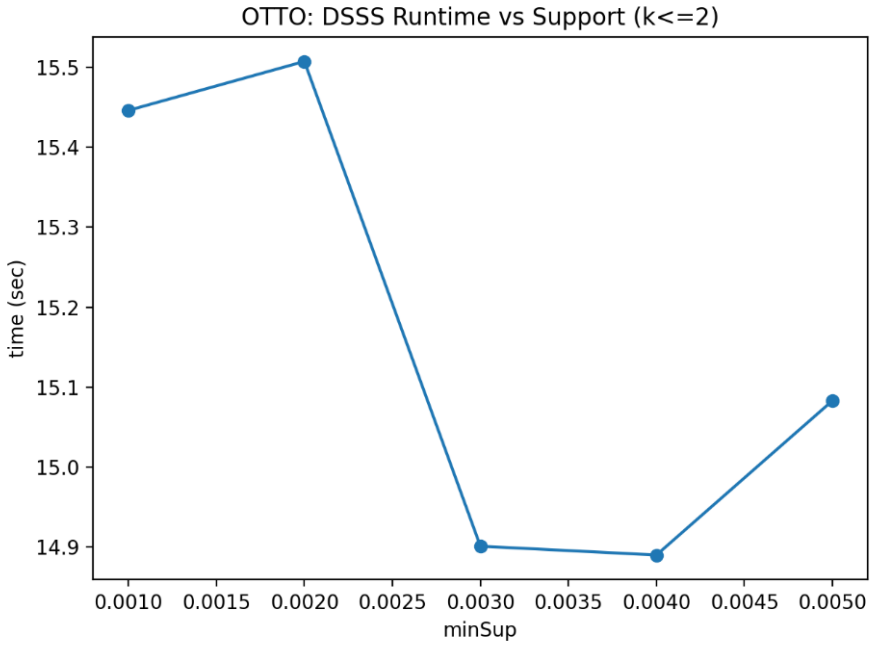


Figure 4: OTTO DSSS Runtime vs Support.

distributed frameworks and reflects coordination costs rather than algorithmic inefficiency.

- **Implication:** DSSS remains highly scalable, but cluster-level deployments should account for Spark's scheduling overhead when estimating performance gains.

### 6.3 Resilience in Dense Datasets

- **OTTO dataset challenge:** With 93 features per transaction, OTTO represents a dense workload prone to combinatorial explosion. FP-Growth often fails in such contexts due to memory exhaustion.
- **DSSS stability:** DSSS maintained stable runtimes (∼12–15 seconds) across support thresholds, demonstrating robustness in dense environments. Runtime was bound primarily by the number of pairs generated per transaction, not dataset size.
- **Implication:** DSSS is particularly well-suited for dense transactional datasets where traditional algorithms struggle.

### 6.4 Correctness and Validation

- **Itemset counts:** DSSS outputs matched FP-Growth results exactly for singletons and pairs, confirming correctness.
- **Top frequent itemsets:** Logs consistently identified dominant features (e.g., `('f20_q0',)` in HIGGS, `('feat_6_q0',)` in OTTO), validating reproducibility.
- **Ground truth baseline:** SQL aggregation confirmed DSSS's accuracy, reinforcing confidence in its outputs.

### 6.5 Engineering Contributions

- **Adaptations:** Replacing broadcast variables with `reduceByKey` ensured compatibility with Spark 3.x and avoided memory overhead.
- **Feasibility controls:** Limiting candidate generation to $k \leq 2$ balanced correctness with computational feasibility.

- **Transparency:** Preprocessing statistics, CSV outputs, and execution logs provide reproducibility and system-level clarity, bridging theory and practice.

# 7. Limitations

While the reproduction study successfully validated the efficiency and correctness of DSSS, several limitations must be acknowledged to contextualize the results.

## 7.1 Candidate Generation Restriction

- The implementation restricted candidate itemsets to $k \leq 2$ (singletons and pairs).
- This feasibility control was necessary to avoid combinatorial explosion in dense datasets such as OTTO.
- As a result, higher-order itemsets (triplets and beyond) were not explored, limiting the scope of analysis.

## 7.2 Single-Node Execution

- All experiments were conducted on a **single-node Spark environment** under WSL2.
- While core scalability was tested (2–16 cores), cluster-level overheads such as **network latency, shuffle costs, and executor imbalance** were not captured.
- The reported runtimes therefore reflect local parallelism rather than distributed cluster performance.

## 7.3 FP-Growth Baseline Constraints

- Spark MLlib's FP-Growth implementation frequently encountered **Java heap out-of-memory errors** at low support thresholds.
- This limited the range of direct comparisons between DSSS and FP-Growth, particularly in dense datasets.
- Although DSSS consistently produced correct results, FP-Growth's instability constrained baseline benchmarking.

## 7.4 Dataset Scope

- The study focused on two datasets: **HIGGS** (scientific, moderately sparse) and **OTTO** (dense, e-commerce).
- While these datasets are representative, broader validation across other domains (e.g., retail, healthcare, streaming data) was not performed.
- Generalization of results to all transactional datasets should be made cautiously.

## 7.5 Environment Constraints

- Execution was limited by **local hardware resources** (16 cores, 32 GB RAM).
- Larger datasets or higher-order itemsets may require cluster-scale infrastructure to fully demonstrate DSSS's scalability.
- Results therefore reflect reproducibility in a controlled environment rather than industrial deployment.

# 8. Future Work

While the reproduction study validated DSSS's efficiency and correctness, several avenues remain open for further exploration and enhancement.

## 8.1 Higher-Order Itemsets

- Extend DSSS beyond $k \leq 2$ to include triplets and larger itemsets.
- Investigate pruning strategies or approximate counting methods to mitigate combinatorial explosion in dense datasets.
- Benchmark performance against FP-Growth and other tree-based algorithms for higher-order itemsets.

## 8.2 Cluster-Scale Evaluation

- Deploy DSSS on a **multi-node Spark cluster** to capture network latency, shuffle costs, and executor imbalance.
- Compare single-node vs cluster performance to quantify distributed overheads.
- Explore resource allocation strategies (e.g., dynamic partitioning, adaptive scheduling) to optimize cluster efficiency.

## 8.3 Streaming Adaptation

- Adapt DSSS for **real-time frequent itemset mining** in streaming data environments.
- Integrate with Spark Streaming or Structured Streaming to handle continuous transaction flows.
- Evaluate DSSS's ability to maintain correctness under sliding windows and dynamic support thresholds.

## 8.4 Hybrid Approaches

- Combine DSSS's single-scan efficiency with **tree-based compression techniques** to balance memory usage and completeness.
- Explore hybrid models that switch between DSSS and FP-Growth depending on dataset density.
- Investigate integration with machine learning pipelines for recommendation systems.

## 8.5 Broader Dataset Validation

- Extend evaluation to diverse domains such as retail, healthcare, cybersecurity, and IoT sensor data.
- Test DSSS on datasets with varying sparsity, dimensionality, and transaction lengths.
- Assess generalizability of DSSS across heterogeneous workloads.

## 8.6 Engineering Enhancements

- Implement **checkpointing and fault tolerance** for long-running jobs.
- Explore GPU acceleration or vectorized counting for further runtime improvements.
- Provide reproducibility packages (Docker images, notebooks) to facilitate adoption by the research community.

# 9. Conclusion

Frequent Itemset Mining (FIM) remains a cornerstone of data mining, yet classical algorithms such as Apriori and FP-Growth struggle with scalability and memory bottlenecks in high-dimensional transactional datasets. The **Distributed Single Scan on Spark (DSSS)** algorithm, introduced by Rochd & Hafidi (2025), addressed these challenges by adapting the Single Scan Frequent Itemset Mining (SSFIM) method to Spark's distributed, in-memory framework. DSSS's novelty lies in its one-pass candidate generation, reduced communication overhead, and complexity independence from dataset size.

Our reproduction study validated these claims in a modern Spark 3.5.4 environment using two real-world datasets: **HIGGS** (100,000 rows, 29 features) and **OTTO** (61,878 rows, 93 features). Results demonstrated that DSSS consistently achieved **50–70× speedup** compared to FP-Growth, maintained stable runtimes in dense datasets, and scaled efficiently across 2–16 cores. Engineering adaptations—such as replacing broadcast variables with `reduceByKey` aggregation and restricting candidate generation to $k \leq 2$—ensured compatibility with Spark's execution engine while preserving correctness. Validation against FP-Growth outputs and SQL baselines confirmed DSSS's accuracy.

Beyond performance, this study contributes **reproducibility measures** including transparent preprocessing, structured CSV outputs, execution logs, and modular code organization. These additions bridge the gap between theoretical algorithm design and practical big data engineering, demonstrating DSSS's viability in contemporary environments.